МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ

Ордена трудового Красного Знамени федеральное государственное бюджетное

образовательное учреждение высшего образования

«Московский технический университет связи и информатики»

Кафедра «Программная инженерия»

Отчет по лабораторной работе 6

Работа с коллекциями

Выполнила: студентка группы БПИ2401

Садовникова Евгения

Проверил: Харрасов Камиль Раисович

Москва, 2025

Содержание

Цель работы 3

Задания 3

Ход работы: 4

Задание 1 4

Листинг 1. class TopWords 4

Выходные данные: 5

Задание 2 6

Листинг 2. class PasswordVerification 6

Листинг 3. class MainStack 7

Выходные данные: 7

Задание 3 7

Листинг 4. class SalesStore 7

Листинг 5. class Product 8

Листинг 6. class MainStore 9

Выходные данные: 9

Контрольные вопросы 10

Вывод 15

Ссылка на GitHub: 15

Цель работы:

Изучить коллекции

Задания

Задание 1.

Написать программу, которая считывает текстовый файл и выводит на экран топ-10 самых часто встречающихся слов в этом файле. Для решения задачи использовать коллекцию Map, где ключом будет слово, а значением — количество его повторений в файле.

Задание 2.

Написать обобщенный класс Stack< T> , который реализует стек на основе массива. Класс должен иметь методы push для добавления элемента в стек, pop для удаления элемента из стека и peek для получения верхнего элемента стека без его удаления.

Задание 3. Поиск заглавной буквы после строчной

Разработать программу для учета продаж в магазине. Программа должна позволять добавлять проданные товары в коллекцию, выводить список проданных товаров, а также считать общую сумму продаж и наиболее популярный товар. Использовать TreeSet для хранения списка проданных товаров.

Ход работы:

Задание 1

Создадим класс TopWords с методом main, в котором будем считывать текстовый файл и выводить на экран топ-10 самых часто встречающихся слов в этом файле (листинг 1).

Листинг 1. class TopWords

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.List;
import java.util.Map;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.HashMap;
import java.util.Scanner;

public class TopWords {
    public static void main(String[] args) {
        // указываем путь к файлу
        String filePath = "text.txt";
        // создаем объект File
        File file = new File(filePath);
        // создаем объект Scanner для чтения файла
        Scanner scanner = null;
        try {
            scanner = new Scanner(file);
        } catch (FileNotFoundException e) {
            e.printStackTrace();
        }
        Map<String, Integer> wordCountMap = new HashMap<>();

        // читаем файл по словам и добавляем их в Map
        while (scanner.hasNext()) {
            String word = scanner.next().toLowerCase();
            if (!word.isEmpty()) {
                                                wordCountMap.put(word,
wordCountMap.getOrDefault(word, 0) + 1);
            }
        }
        scanner.close();
        // создаем список из элементов Map
                List<Map.Entry<String,   Integer>>   list   =   new
ArrayList<>(wordCountMap.entrySet());

        // сортируем список по убыванию количества повторений
           Collections.sort(list,  new  Comparator<Map.Entry<String,
Integer>>() {
```

```
                @Override
                public int compare(Map.Entry<String, Integer> o1,
                        Map.Entry<String, Integer> o2) {
                    return o2.getValue().compareTo(o1.getValue());
                }
            });
            /*
               * new Comparator<Map.Entry<String, Integer>>() { ... }
Создаём анонимный объект
              * Comparator, который определяет правило сравнения двух
элементов списка.
             *
             * public int compare(...)
             * Метод, который сравнивает две пары (o1 и o2)
             *
             * Map.Entry<String, Integer> o1, o2
              * Два элемента списка, которые нужно сравнить. Каждый —
пара слово →
             * количество.
             *
             * o2.getValue()
             * Берём количество второго слова.
             * .compareTo(o1.getValue())
              * Сравниваем его с количеством первого слова. Возвращает:
<0, 0 или >0.
             */

            // выводим топ-10 слов
            int limit = Math.min(10, list.size());
            for (int i = 0; i < limit; i++) {
                Map.Entry<String, Integer> entry = list.get(i);
                        System.out.println(entry.getKey()  +  ":  "  +
entry.getValue());
            }
        }
    }
```

Выходные данные:

Работа программы представлена на рисунке 1.

```
PS C:\Users\Evgen\Desktop\ИТИП> java TopWords.java
will: 2
also: 2
is: 2
the: 2
there: 2
characters,: 2
but: 1
no: 1
capital: 1
be: 1
PS C:\Users\Evgen\Desktop\ИТИП>
ebugX    Java: Ready
```

Рисунок 1. Результат программы

Задание 2

Создадим класс Stack с методами pop, push peek (листинг 2). Запуск представлен в классе MainStack (листинг 3)

Листинг 2. class PasswordVerification

```
import java.util.*;

public class Stack<T> {
    private ArrayList<T> data;

    public Stack() {
        data = new ArrayList<>();
    }

    public Stack(int capacity) {
        data = new ArrayList<>(capacity);
    }

    public void push(T element) {
        data.add(element);
    }

    public T pop() {
        if (data.isEmpty()) {
            throw new IllegalStateException("Stack underflow");
        }
        T res = data.get(data.size() - 1);
        data.remove(data.size() - 1);
        return res;
    }

    public T peek() {
        if (data.isEmpty()) {
            throw new IllegalStateException("Stack underflow");
```

```
        }
        return data.get(data.size() - 1);
    }
}
```

Листинг 3. class MainStack

```
public class MainStack {
    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<>(10);
        stack.push(1);
        stack.push(2);
        stack.push(3);
        System.out.println(stack.pop());
        System.out.println(stack.peek());
        stack.push(4);
        System.out.println(stack.pop());
    }
}
```

Выходные данные:

Работа программы представлена на рисунке 2.

```
PS C:\Users\Evgen\Desktop\ИТИП> java MainStack.java
3
2
4
PS C:\Users\Evgen\Desktop\ИТИП>
```

Рисунок 2. Результат программы

Задание 3

Создадим класс SalesStore и Product для учета продаж в магазине. (листинг 4–5). Запуск представлен в классе MainStore (листинг 6)

Листинг 4. class SalesStore

```
public class SalesStore {
    private TreeSet<Product> product;

    public SalesStore() {
        product = new TreeSet<>();
    }

    public void addProduct(String name, double price, int count) {
        for (Product p : product) {
            if (p.getName().equalsIgnoreCase(name)) {
                p.sell(price, count);
                return;
            }
        }
        Product p = new Product(name, price);
        p.sell(price, count);
        product.add(p);
```

```
    }

    public TreeSet<Product> setProducts() {
        return product;
    }

    public double sumSales() {
        double sum = 0;
        for (Product p : product) {
            sum += p.getPrice() * p.getCount();
        }
        return sum;
    }

    public Product popularProduct() {
        if (product.isEmpty())
            return null;
        Product popular = product.first();
        for (Product p : product) {
            if (p.getCount() > popular.getCount()) {
                popular = p;
            }
        }
        return popular;
    }

}
```

Листинг 5. class Product

```
public class Product implements Comparable<Product> {
    private String name;
    private double price;
    private int count;

    public Product(String name, double price) {
        this.name = name;
        this.price = price;
        this.count = 0;
    }

    public String getName() {
        return name;
    }

    public double getPrice() {
        return price;
    }

    public int getCount() {
        return count;
    }

    public void sell(double price, int count) {
        if (count <= 0) {
```

```
                    throw new IllegalArgumentException("Количество должно
быть положительным");
                }
                if (price <= 0) {
                     throw new IllegalArgumentException("Цена должна быть
положительной");
                }
                 this.price = (this.price * this.count + price * count) /
(this.count + count);
                this.count += count;
            }

            @Override
            public String toString() {
                return name;
            }

            @Override
            public int compareTo(Product other) {
                return this.name.compareTo(other.name);
            }
        }
```

Листинг 6. class MainStore

```
        public class MainStore {
            public static void main(String[] args) {
                SalesStore store = new SalesStore();
                store.addProduct("Яблоки", 100, 5);
                store.addProduct("Бананы", 80, 9);
                store.addProduct("Груши", 1, 8);
                store.addProduct("Груши", 3, 8);
                store.addProduct("Груши", 3, 9);
                System.out.println(store.setProducts());
                System.out.println(store.sumSales());
                System.out.println(store.popularProduct());
            }
        }
```

Выходные данные:

Работа программы представлена на рисунке 3.

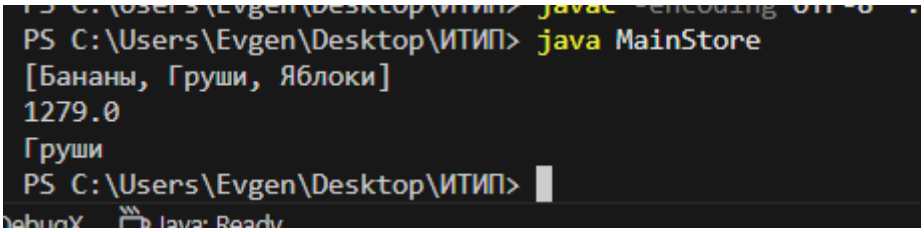

Рисунок 3. Результат программы

Контрольные вопросы

1. Какие интерфейсы коллекций есть в Java?

В языке Java интерфейсы коллекций определены в пакете java.util и образуют иерархию, позволяющую унифицированно работать с различными типами данных. Основные интерфейсы:

Collection – базовый интерфейс для всех коллекций, кроме Map. Он задает базовые методы добавления, удаления, проверки наличия элементов и итерации.

List – упорядоченная коллекция, допускающая хранение дубликатов и доступ по индексу.

Set – коллекция, не допускающая дубликатов, может быть упорядоченной (TreeSet) или сохранять порядок добавления (LinkedHashSet).

Queue – коллекция, поддерживающая структуру «очередь» с операциями вставки и извлечения элементов в определенном порядке (FIFO или с приоритетом).

Deque – двусторонняя очередь, позволяющая добавлять и удалять элементы с обеих сторон.

Map – отдельный интерфейс для отображений «ключ–значение», где ключи уникальны, а значения могут повторяться.

2. Какие классы коллекций есть в Java?

Классические реализации коллекций включают:

ArrayList и **LinkedList** – динамические списки, реализующие интерфейс List.

HashSet, **LinkedHashSet**, **TreeSet** – различные реализации интерфейса Set.

HashMap, **LinkedHashMap**, **TreeMap**, **Hashtable** – реализации интерфейса Map.

Vector и **Stack** – устаревшие синхронизированные структуры данных.

PriorityQueue и **ArrayDeque** – очереди и двусторонние очереди.

Эти классы различаются по производительности, внутренней структуре данных и поддерживаемой синхронизации.

3. Что такое итератор?

Итератор (Iterator) – это объект, предоставляющий последовательный доступ к элементам коллекции без раскрытия внутренней структуры данных. Итератор реализует интерфейс Iterator<E> и поддерживает методы:

boolean hasNext() – проверяет, есть ли следующий элемент для обхода.

E next() – возвращает следующий элемент коллекции.

void remove() – удаляет текущий элемент из коллекции.

Итераторы повышают безопасность и абстрагируют обход коллекций, позволяя работать с любыми типами коллекций одинаково.

3. Как работают коллекции на основе интерфейса Map?

Коллекции, реализующие интерфейс Map, хранят пары «ключ–значение». Каждый ключ уникален, значения могут повторяться. Основные особенности работы:

Добавление элемента: put(key, value) заменяет существующее значение для ключа.

Получение значения: get(key) возвращает значение по ключу.

Удаление: remove(key) удаляет пару по ключу.

Проверка наличия: containsKey(key) и containsValue(value).

Существуют реализации с разными особенностями: HashMap – неупорядоченный, LinkedHashMap – сохраняет порядок вставки, TreeMap – сортирует по ключу.

4. **Как работают коллекции на основе интерфейса List?**

Коллекции List представляют упорядоченные наборы элементов, допускают дубликаты и предоставляют доступ по индексу. Основные операции:

add(element) / add(index, element) – добавление элементов.

get(index) – получение элемента по индексу.

remove(index) / remove(object) – удаление элементов.

indexOf(object) / lastIndexOf(object) – поиск индекса элемента.

Основные реализации: ArrayList (эффективен для случайного доступа), LinkedList (эффективен для вставки и удаления элементов).

5. Как работают коллекции на основе интерфейса Set

Коллекции Set обеспечивают хранение уникальных элементов. Основные свойства:

HashSet – неупорядоченный набор, обеспечивает быстрый доступ через хеширование.

LinkedHashSet – сохраняет порядок добавления.

TreeSet – автоматически сортирует элементы по естественному порядку или с помощью Comparator.

Методы: add(element), remove(element), contains(element). Дубликаты автоматически игнорируются.

6. Как можно синхронизировать коллекции в Java?

Синхронизация коллекций необходима для многопоточного доступа. Возможные способы:

Обертки Collections.synchronizedXXX():

List<String> syncList = Collections.synchronizedList(new ArrayList<>());

Concurrent коллекции из пакета java.util.concurrent: ConcurrentHashMap, CopyOnWriteArrayList, обеспечивают потокобезопасный доступ без блокировок.

Синхронизация вручную через synchronized блоки при обращении к коллекции.

7. Какие методы предоставляет интерфейс Collection

Интерфейс Collection определяет базовые методы работы с коллекциями:

add(E e), addAll(Collection<? extends E> c) – добавление элементов.

remove(Object o), removeAll(Collection<?> c), retainAll(Collection<?> c) – удаление или фильтрация элементов.

contains(Object o), containsAll(Collection<?> c) – проверка наличия элементов.

size(), isEmpty(), clear() – работа с размером и очистка коллекции.

iterator() – получение итератора для обхода элементов.

9. Какие реализации интерфейса List вы знаете?

ArrayList – динамический массив, быстрый доступ по индексу, медленные вставки/удаления в середине.

LinkedList – двусвязный список, медленный доступ по индексу, быстрые вставки/удаления.

Stack – расширение Vector, реализует структуру данных стек.

10. Какие реализации интерфейса Set вы знаете?

HashSet – неупорядоченный набор, уникальные элементы.

LinkedHashSet – сохраняет порядок добавления.

TreeSet – автоматически сортирует элементы.

EnumSet – специализированный Set для типов enum.

11. Что такое Comparable и Comparator?

Comparable – интерфейс для определения естественного порядка объектов. Метод compareTo(T o) возвращает отрицательное число, ноль или положительное число.

Comparator – интерфейс для внешнего задания порядка сортировки объектов. Метод compare(T o1, T o2) возвращает отрицательное число, ноль или положительное число.

Пример:

```
class Person implements Comparable<Person> {
    int age;
    public int compareTo(Person p) { return this.age - p.age; }
}
Comparator<Person> nameComparator = (p1, p2) -> p1.name.compareTo(p2.name);
```

12. Что такое параметр типа?

Параметр типа (например, T, E, K, V) используется в дженериках для создания универсальных классов и методов, обеспечивающих **типобезопасность** и возможность работы с любыми типами данных.

13. Как параметризовать метод, класс?

Класс:

```
class Box<T> {
    private T value;
    void set(T value) { this.value = value; }
    T get() { return value; }
}
```

Метод:

```
<T> void print(T item) { System.out.println(item); }
```

14. Что такое стирание типов

Стирание типов – это процесс, при котором информация о параметрах типа удаляется во время компиляции для совместимости с JVM. На этапе выполнения тип параметра заменяется на верхнюю границу (обычно Object), что предотвращает создание массивов конкретного дженерик-типа.

15. Как можно обойти ограничения стирания типов?

Использовать **ограничения типа** (<T extends Number>).

Применять **Class** для создания массивов через Array.newInstance().

Использовать **wildcard типы** и явные приведения типа.

16. Как работают дженерики с массивами?

Создать массив дженериков напрямую нельзя (T[] array = new T[10] вызовет ошибку). Возможные обходные варианты:

```
T[] array = (T[]) new Object[10];
T[] array = (T[]) Array.newInstance(clazz, 10);
```

17. Можно ли создать массив дженериков? Нет, прямое создание невозможно из-за стирания типов, но допускаются обходные варианты через кастинг или Array.newInstance().

18. Что такое wildcard тип? Wildcard (?) – это неизвестный тип параметра дженерика.

<?> – любой тип.

<? extends T> – любой тип, наследующий T (ограничение сверху).

<? super T> – любой тип, являющийся родителем T (ограничение снизу).

19. В чем разница между <extends T> и <super T>?

<extends T> – ограничение сверху, подходит для **чтения элементов** (Producer).

<super T> – ограничение снизу, подходит для **записи элементов** (Consumer).

Правило PECS: **Producer Extends, Consumer Super**.

Вывод

В ходе выполнения лабораторной работы были изучены методы и интерфейсы коллекций.

Ссылка на GitHub: https://github.com/Evgeshhha/Information-technology-and-programming